

BTS SERVICES INFORMATIQUES AUX ORGANISATIONS	SESSION 2026
ANNEXE 9-1-B : Fiche descriptive de réalisation professionnelle (recto)	
Épreuve E6 - Conception et développement d'applications (option SLAM)	

DESCRIPTION D'UNE RÉALISATION PROFESSIONNELLE		N° réalisation : 1
Nom, prénom : Lanllier Arthur		N° candidat : 2541496691
Épreuve ponctuelle ☒	Contrôle en cours de formation ☐	Date : 11 / 11 / 2025
Organisation support de la réalisation professionnelle : Projet personnel / Projet de formation Pokocard Masters.		
Intitulé de la réalisation professionnelle : Développement du back-end API de Pokocard Masters.		
Période de réalisation : 2024 / 2025 Lieu : AFTEC		
Modalité : ☐ Seul(e) ☒ En équipe		
Compétences travaillées ☒ Concevoir et développer une solution applicative Assurer ☒ la maintenance corrective ou évolutive d'une solution applicative Gérer les ☒ données		
Conditions de réalisation¹ (ressources fournies, résultats attendus) Cette réalisation a été réalisée à partir d'un cahier des charges fonctionnel implicite : créer un jeu de tirage de boosters de cartes Pokémon, avec une authentification des joueurs, un tirage limité à un booster par jour, un système de rareté des cartes et une collection associée à chaque compte. Les résultats attendus : - La mise en place d'une API REST permettant de gérer l'authentification, les tirages quotidiens, la gestion des cartes et des collections. - La création d'une base de données robuste et cohérente pour stocker les informations des joueurs, des cartes, des tirages et des raretés. - La fourniture d'une interface technique claire et documentée pour le front-end mobile. (Expo Go + NativeWind).		

¹ En référence aux *conditions de réalisation et ressources nécessaires* du bloc « Conception et développement d'applications » prévues dans le référentiel de certification du BTS SIO.

Description des ressources documentaires, matérielles et logicielles utilisées²

Les principales ressources utilisées pour ce projet sont :

- Le Framework Django, avec l'API Django Ninja, pour le développement du back-end.
- La base de données MariaDB, gérée via Docker pour la persistance des données.
- Git et GitHub pour le versionnage du code et la collaboration.
- Des outils de développement comme VS Code pour développement ainsi que tester les routes API.
- La documentation officielle de Django, Django Ninja, MariaDB ainsi que Docker.

Modalités d'accès aux productions³ et à leur documentation⁴

La production de ce projet est accessible de la manière suivante :

- Le code source du back-end est disponible sur le repository GitHub à l'URL suivante : https://github.com/PokeCard-Masters/pokecard_masters
- Le dossier back-end du projet est clairement identifié dans l'arborescence du projet.
- Le serveur de démonstration pour jouer à l'application est sur l'URL pokecardmasters.lebaron.sh/
- Le schéma de la base de données, les scripts, et la documentation de l'API Swagger générée par Django Ninja est incluse dans le dépôt ainsi que dans ce dossier.

suivante :

BTS SERVICES INFORMATIQUES AUX ORGANISATIONS

SESSION 2026

**ANNEXE 9-1-B : Fiche descriptive de réalisation professionnelle
(verso, éventuellement pages suivantes)**

Épreuve E6 - Conception et développement d'applications (option SLAM)

² Les réalisations professionnelles sont élaborées dans un environnement technologique conforme à l'annexe II.E du référentiel du BTS SIO.

³ Conformément au référentiel du BTS SIO « Dans tous les cas, les candidats doivent se munir des outils et ressources techniques nécessaires au déroulement de l'épreuve. Ils sont seuls responsables de la disponibilité et de la mise en œuvre de ces outils et ressources. La circulaire nationale d'organisation précise les conditions matérielles de déroulement des interrogations et les

pénalités à appliquer aux candidats qui ne se seraient pas munis des éléments nécessaires au déroulement de l'épreuve. ». Les éléments peuvent être un identifiant, un mot de passe, une adresse réticulaire (URL) d'un espace de stockage et de la présentation de l'organisation du stockage.

⁴ Lien vers la documentation complète, précisant et décrivant, si cela n'a été fait au verso de la fiche, la réalisation professionnelle, par exemples service fourni par la réalisation, interfaces utilisateurs, description des classes ou de la base de données.

Descriptif de la réalisation professionnelle, y compris les productions réalisées et schémas explicatifs

Pokecard Masters est une application de jeu de collection de cartes Pokémon, permettant aux joueurs d'ouvrir un booster par jour et d'obtenir 10 cartes aléatoires selon des règles de rareté prédéfinies. Le back-end assure la logique métier centrale du jeu, en gérant l'authentification des joueurs, les tirages quotidiens, la sauvegarde des cartes dans les collections personnelles et l'exposition d'une API REST pour le front-end mobile.

Dans le cadre de ma formation BTS SIO option SLAM, j'ai conçu ce projet pour démontrer mes compétences en développement d'API et de gestion de données. Les besoins fonctionnels étaient clairs :

- Authentifier les joueurs et gérer leurs sessions.
- Permettre un tirage unique de booster par jour, avec 10 cartes aléatoires tirées d'un catalogue Pokémon.
- Stocker les cartes obtenues dans une collection associée à chaque joueur, en respectant les niveaux de rareté (commune, rare, etc.).
- Fournir une API REST robuste et documentée, consommable par une application mobile Expo Go. Le public cible est les collectionneurs de cartes Pokémon, cherchant une expérience quotidienne simple et addictive.

L'architecture retenue repose sur Django avec Django Ninja pour l'API, connectée à une base MariaDB conteneurisée via Docker. Les principales fonctions sont :

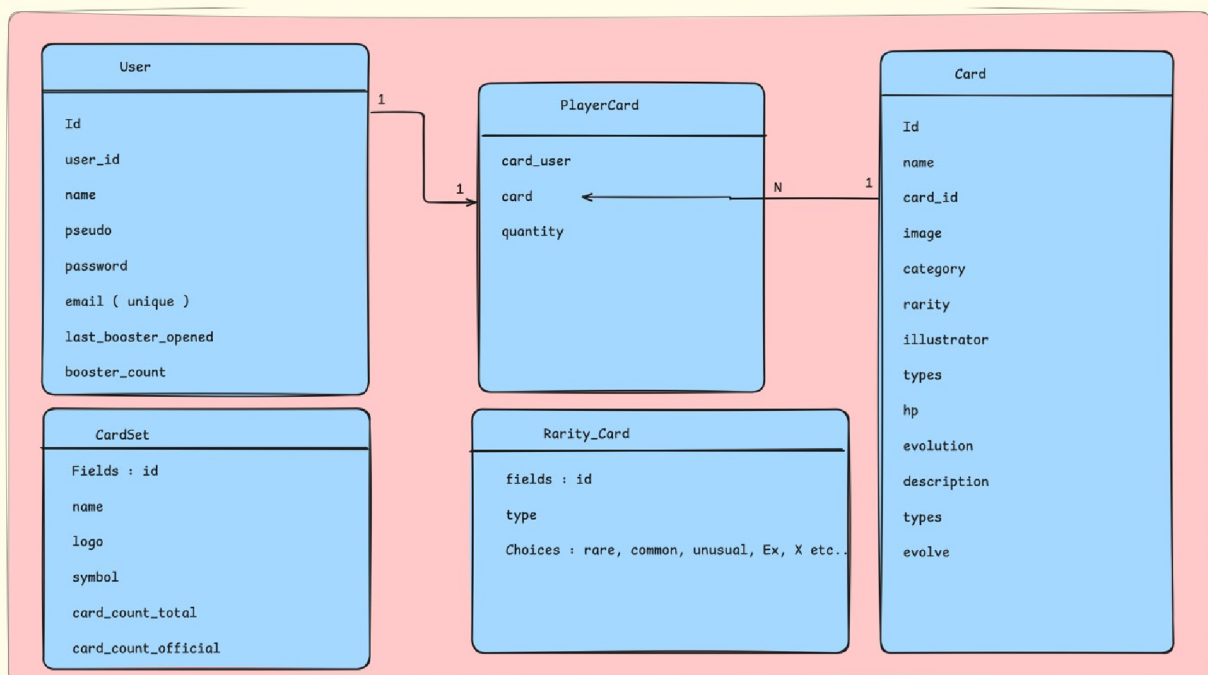
- Modélisation de la base avec des entités Utilisateur, Carte (avec champs nom, image, rareté), Tirage (date, booster_id), Collection (lien utilisateur-carte).
- Routes API : /auth/login , /daily-booster , /booster/open , /user/collection/pagination, /cards .
- Logique métier : vérification du tirage quotidien, algorithme de rareté pondérée, persistance des résultats. L'ensemble est déployé localement via Docker Compose pour un environnement reproductible.

J'ai réalisé les éléments suivants :

- Modélisation de la base de données : Création des modèles Django pour les entités mentionnées, avec migrations et relation ForeignKey.
- Développement des routes API : implémentation des endpoints principaux avec Django Ninja, incluant validation des données, gestion d'erreurs et authentification JWT.
- Logique métier : Algorithme de tirage aléatoire pondéré par rareté (ex : 70% communes, 20% rares, 10% de légendaires), restriction à un tirage/jour via timestamp.

Schéma explicatif de la base de données :

Schéma explicatif de la base de données :



Tables principales et leurs rôles :

- User : Gestion des joueurs (email unique, mot de passe, date dernier booster, compteur de booster).
- Card : Catalogue des cartes Pokémon.
- PlayerCard : Table d'association centrale reliant chaque joueur à ses cartes possédées.
- CardSet : Extensions / séries de cartes (nom, logo, nombre total de cartes).
- Rarity_Card : Référentiel des raretés (Common, Rare etc..).

Relations fonctionnelles clés :

- Un joueur possède plusieurs PlayerCard.
- Une carte est référencée dans plusieurs PlayerCard.
- PlayerCard.quantity gère les multiples exemplaires d'une même carte dans une collection.

Productions réalisées :

- 5 modèles Django avec ForeignKey, choices et migrations.
- Base de données MariaDB conteneurisée Docker.
- Script d'initialisation pour importer le catalogue Card.
- Relations testées pour garantir l'intégrité des collections.

Ce schéma assure la persistance cohérente des données de jeu et permet à l'API Django Ninja de calculer les collections, vérifier les droits de tirage et renvoyer les données JSON au front-end mobile Expo Go.

Pour finir, voici quelques captures d'écran de mon arborescence de fichiers pour le côté back-end, ainsi que quelques images de la base de données en terminal de commandes :

--

MediaDB (pokemon)> select * from esp_card;

id	name	card_id	image	hp	category	rarity	illustrator	types	description
		evolution evolve_from							
1	1 Spigafillo	esp-001	https://assets.tigdex.net/en/es/esp/001		Pokemon	None	You Nishida	["Grass"]	NULL
2	2 Seedale	sv18-00-001	https://assets.tigdex.net/en/es/sv18.00/001		Pokemon	Common	Oswaldo KATO	["Grass"]	NULL
3	3 Venusaur ex	sv07-001	https://assets.tigdex.net/en/es/sv07/001		Pokemon	Double rare	Saiki Hayashiro	["Grass"]	NULL
4	4 Oddish	sv03-001	https://assets.tigdex.net/en/es/sv03/001		Pokemon	Common	Midori Harada	["Grass"]	NULL
5	5 Joltik	sv04-S-001	https://assets.tigdex.net/en/es/sv04.S/001		Pokemon	Common	Naoya Kimura	["Grass"]	NULL
6	6 Surskit	sv04-S-001	https://assets.tigdex.net/en/es/sv04.S/001		Pokemon	Common	Yoriyuki Ikegami	["Grass"]	NULL
7	7 Scyther	sv00-001	https://assets.tigdex.net/en/es/sv00/001		Pokemon	Common	Douke	["Grass"]	NULL
8	8 Pineco	sv01-001	https://assets.tigdex.net/en/es/sv01/001		Pokemon	Common	Shigenori Nagishi	["Grass"]	NULL
9	9 Eggspurt	sv08-001	https://assets.tigdex.net/en/es/sv08/001		Pokemon	Common	Tetsu Kayama	["Grass"]	NULL
10	10 Tangela	sv04-001	https://assets.tigdex.net/en/es/sv04/001		Pokemon	Common	Yoriyuki Ikegami	["Grass"]	NULL
11	11 Ethan's Pincir	sv01-001	https://assets.tigdex.net/en/es/sv01/001		Pokemon	Uncommon	GIDORA	["Grass"]	NULL
12	12 Snivy	sv10-S0-001	https://assets.tigdex.net/en/es/sv10.00/001		Pokemon	Common	Susumu Maeya	["Grass"]	NULL
13	13 Hoppip	sv02-001	https://assets.tigdex.net/en/es/sv02/001		Pokemon	Common	Yukiko Baba	["Grass"]	NULL
14	14 Caterpie	sv00-001	https://assets.tigdex.net/en/es/sv00/001		Pokemon	Common	Shinichi Yukichi	["Grass"]	NULL
15	15 Eggspurt	sv00-001	https://assets.tigdex.net/en/es/sv00.S/001		Pokemon	Common	Ieri Suzuki	["Grass"]	NULL
16	16 Pineco	sv01-S-001	https://assets.tigdex.net/en/es/sv01.S/001		Pokemon	Common	Kouki Saitou	["Grass"]	NULL
17	17 Bulbasaur	sv01-S-001	https://assets.tigdex.net/en/es/sv01.S/001		Pokemon	Common	You Nishida	["Grass"]	NULL
18	18 Blaine's Azumarie	gym-01	https://assets.tigdex.net/en/gym/gym01/1		Pokemon	Rare Holo	Ken Sugimori	["Fire"]	NULL
19	19 Butterfree V	sv01-S	https://assets.tigdex.net/en/es/01/sv01/1		Pokemon	Holo Rare V	Saiki Hayashiro	["Grass"]	NULL
20	20 Ho-Oh	sv01-01	https://assets.tigdex.net/en/es/01/sv01/1		Pokemon	Rare	Kouki Saitou	["Fire"]	It will reveal itself before a pure-hearted Trainer by shining its bright, rainbow-colored wings.
21	21 Beedrill V	sv01H-001	https://assets.tigdex.net/en/es/01/sv01H/001		Pokemon	Holo Rare V	NULL	["Grass"]	NULL
22	22 Caterpie	sv01-S	https://assets.tigdex.net/en/es/01/sv01/1		Pokemon	Common	uta	["Grass"]	For protection, it releases a horrible stench from the antenna on its head to drive away enemies.
23	23 Weavile	sv01-S	https://assets.tigdex.net/en/es/01/sv01/1		Pokemon	Common	sui	["Grass"]	Its poison stinger is very powerful. Its bright-colored body is intended to warn off its enemies.
24	24 Weavile	sv01-S	https://assets.tigdex.net/en/es/01/sv01/1		Pokemon	Common	Kanetsuke Sakuma	["Grass"]	Its poison stinger is very powerful. Its bright-colored body is intended to warn off its enemies.
25	25 Bellspurt	sv01-S	https://assets.tigdex.net/en/es/01/sv01/1		Pokemon	Common	Bakio	["Grass"]	Its bud looks like a human face. Because of the bud, it is rumored to be a type of legendary mandrake plant.
26	26 Eggspurt	sv01H-001	https://assets.tigdex.net/en/es/01/sv01H/001		Pokemon	NULL	NULL	["Grass"]	NULL
27	27 Calabi V	sv01-S	https://assets.tigdex.net/en/es/01/sv01/1		Pokemon	Holo Rare V	PLANETA Igarashi	["Grass"]	NULL
28	28 Yama	sv01-S	https://assets.tigdex.net/en/es/01/sv01.S/1		Pokemon	Common	Kouki Saitou	["Grass"]	If it flaps its wings really fast, it can generate shock waves that will shatter windows in the area.
29	29 Team Magma's Numel	dc1-S	https://assets.tigdex.net/en/es/dc1/1		Pokemon	Common	Akira Kameyama	["Fire"]	NULL
30	30 Venusot	sv01H-001	https://assets.tigdex.net/en/es/01/sv01H/001		Pokemon	Common	NULL	["Grass"]	NULL
31	31 Hoppip	sv01-S	https://assets.tigdex.net/en/es/01/sv01/1		Pokemon	Common	Kouki Saitou	["Grass"]	NULL

Database changed

MariaDB [pokemon]> show tables;

Tables_in_pokemon
app_card
app_cardset
app_playercard
app_rarity_card
app_user
auth_group
auth_group_permissions
auth_permission
auth_user
auth_user_groups
auth_user_user_permissions
django_admin_log
django_content_type
django_migrations
django_session

15 rows in set (0.001 sec)

MariaDB [pokemon]> select * from app_playercard;

id	quantity	card_id	card_user_id
1	1	3119	1
2	2	3420	1
3	1	275	1
4	1	448	1
5	1	3590	1
6	1	2421	1
7	2	586	1
8	1	2877	1
9	1	411	1
30	1	9956	1
31	1	4144	1
32	2	10378	1
33	1	4420	1
34	1	1198	1
35	1	8385	1
36	3	4275	1
37	1	7274	1
38	2	10245	1
39	2	9825	1
40	1	1987	1
41	1	4832	1
42	1	7979	1
43	1	8781	1
44	1	4414	1
45	1	5345	1
46	1	5797	1
47	1	3298	1
48	1	10626	1
49	1	10089	1
50	1	5383	1
51	2	3997	1
52	2	1351	1


```

arthur@Mac ~ %
arthur@Mac ~ % docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
bf4725130c11   mariadb:10.11 "docker-entrypoint.s..." 12 days ago   Exited (0) 4 days ago           xefi
f94684e91f5b   mariadb:latest "docker-entrypoint.s..." 2 weeks ago   Exited (0) 3 days ago           pokemon
arthur@Mac ~ %
arthur@Mac ~ % docker start f9
f9
arthur@Mac ~ %
arthur@Mac ~ % docker exec -it f9 /bin/bash
root@f94684e91f5b:/# mariadb -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 3
Server version: 12.2.2-MariaDB-ubu2404 mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

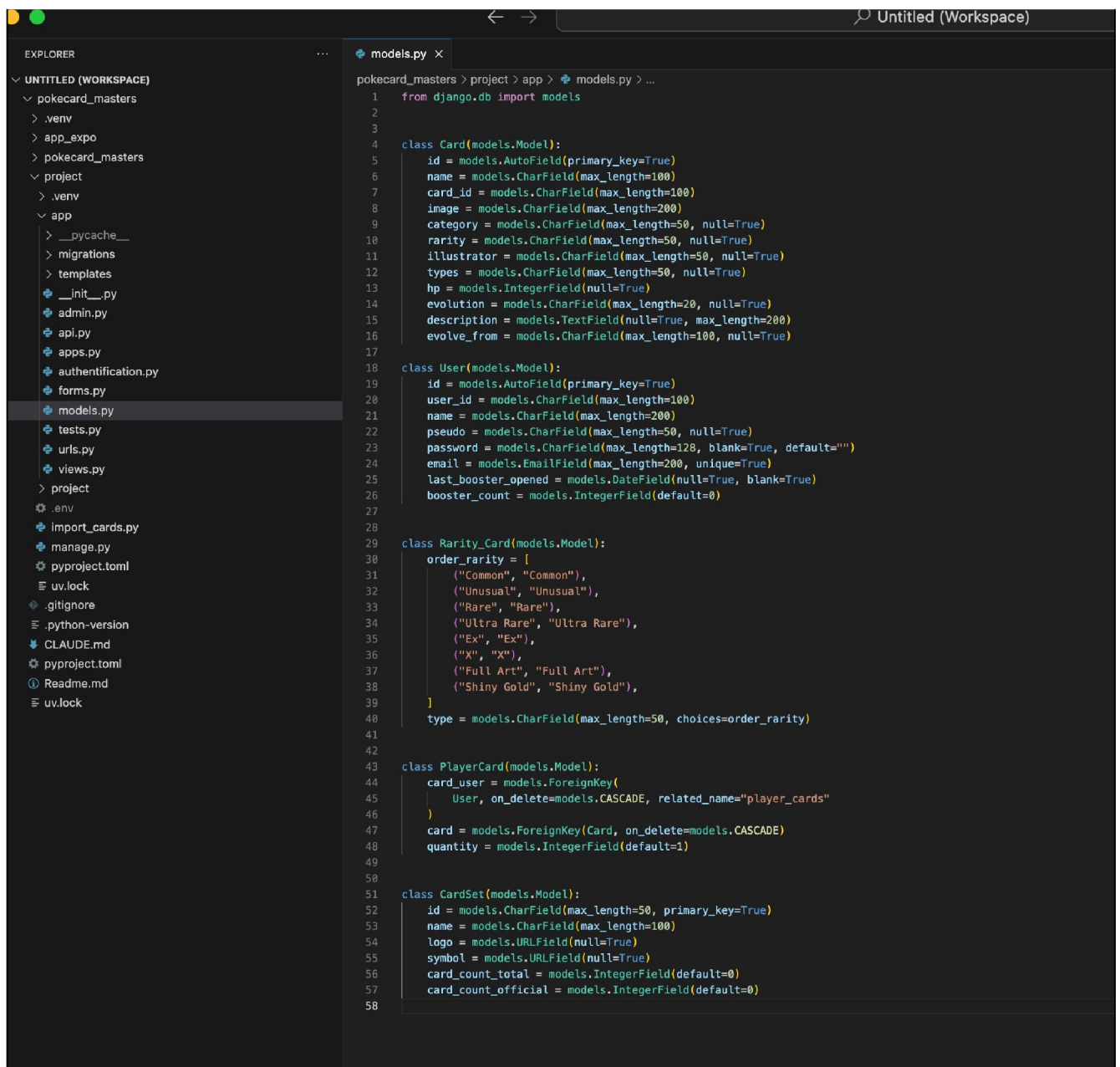
MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| pokemon |
| sys |
+-----+
5 rows in set (0.006 sec)

MariaDB [(none)]> _

```

```
> .venv
> app_expo
> pokecard_masters
✓ project
  > .venv
  ✓ app
    > __pycache__
    > migrations
    > templates
    🔄 __init__.py
    🔄 admin.py
    🔄 api.py
    🔄 apps.py
    🔄 authentication.py
    🔄 forms.py
    🔄 models.py
    🔄 tests.py
    🔄 urls.py
    🔄 views.py
```

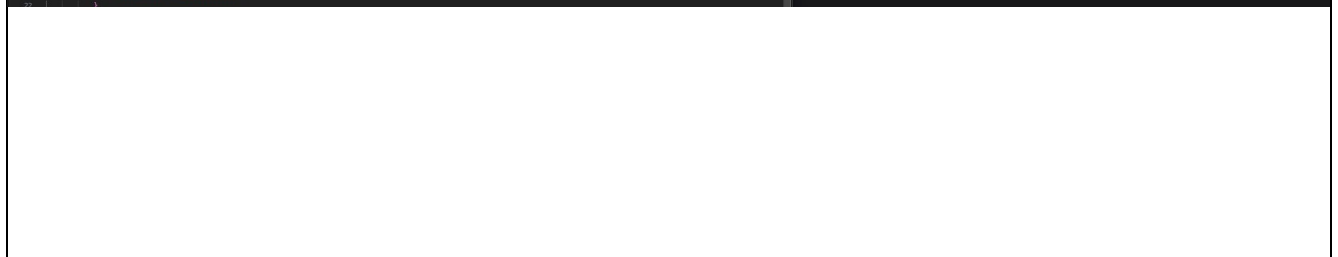
```
✓ project
  > __pycache__
  🔄 __init__.py
  🔄 asgi.py
  🔄 settings_test.py
  🔄 settings.py
  🔄 urls.py
  🔄 wsgi.py
  ⚙️ .env
  🔄 import_cards.py
  🔄 manage.py
  ⚙️ pyproject.toml
  ≡ uv.lock
  💎 .gitignore
  ≡ .python-version
  ⚡ CLAUDE.md
  ⚙️ pyproject.toml
  ⓘ Readme.md
  ≡ uv.lock
```



Ces photos montrent donc la structure du projet. Nous avons donc le dossier 'app' qui regroupe la logique métier et project qui elle recense la configuration Django. Le projet inclut les fichiers standards :

- Models.py
- Views.py
- Urls.py
- Forms.py
- Authentification.py

Ensuite nous avons le modèles de données expliqué plus haut dans le dossier ainsi que la base de données Docker + MariaDB. J'ai effectué des requêtes pour montrer les données inscrites en base. Pour finir je vous joints des photos de la documentation Swagger ainsi qu'une photo d'un des endpoints de l'API.



default

^

GET	/api/hello	Hello	✓
GET	/api/card	List Card	✓
GET	/api/me	Get Me	✓
GET	/api/player/card	View Card	✓
POST	/api/auth/register	Register	✓
POST	/api/auth/login	Login	✓
POST	/api/auth/change-password	Change Password	✓

